SFWRENG / MECHTRON 3K04 Software Development

Today: Precise Specification and Tabular Expressions

McMaster University
Engineering
Computing & Software

October 4th, 2026

Dr. Angela Zavaleta Bernuy

Plan Review: Requirements & Validation

- If we don't get the requirements right, it does not matter how good we build our system.
- The world of *requirements*, can be divided into a few parts:
  - **Eliciting** requirements: how do we **capture** the requirements
  - **Documenting** requirements: how do we **keep track** of the requirements
    - **Formal specifications**: well-defined syntax and semantics (usually mathematical)
    - **The 4 variable model**: relationship between requirements and design
  - How do we know if we elicit and document requirements **correctly**?
- *Validation*: determining whether or not we got the *right* requirements
  - Not to be confused with *verification*, which is determining whether or not we built the system *right* (i.e., if it complies with the requirements)
  - We often refer to Validation and Verification as V & V.



*** WE ARE HERE ***

Deliverable 2 Map

D2 Document Section	W5 L1	W5 L2	W5 L3	W6
1. Introduction and context	-	-	-	-
2. Precise specification	X	X		
3. Modular design and interfaces		X	X	
3.1. External ADC <-> Monitoring interface				X
4. Implementation notes				X
5. Traceability table				X
6. Assumptions and decisions			X	X

Formal Specifications

- Specifications are central to many aspects of software engineering
- A *specification* is just a precise description of behaviour
- *Requirements specifications* describe required behaviour in a system
- *Design specifications* describe the behaviour prescribed in a design

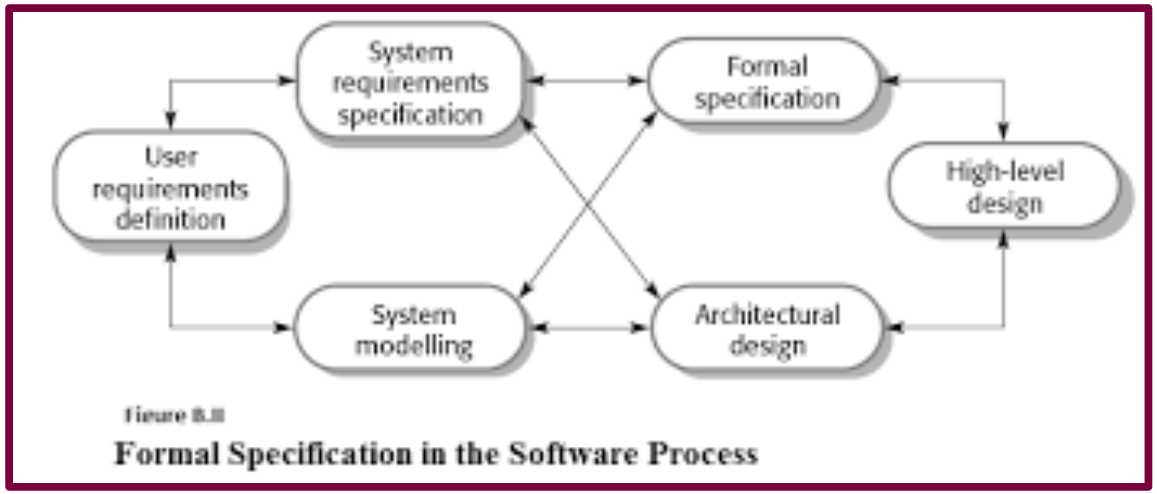
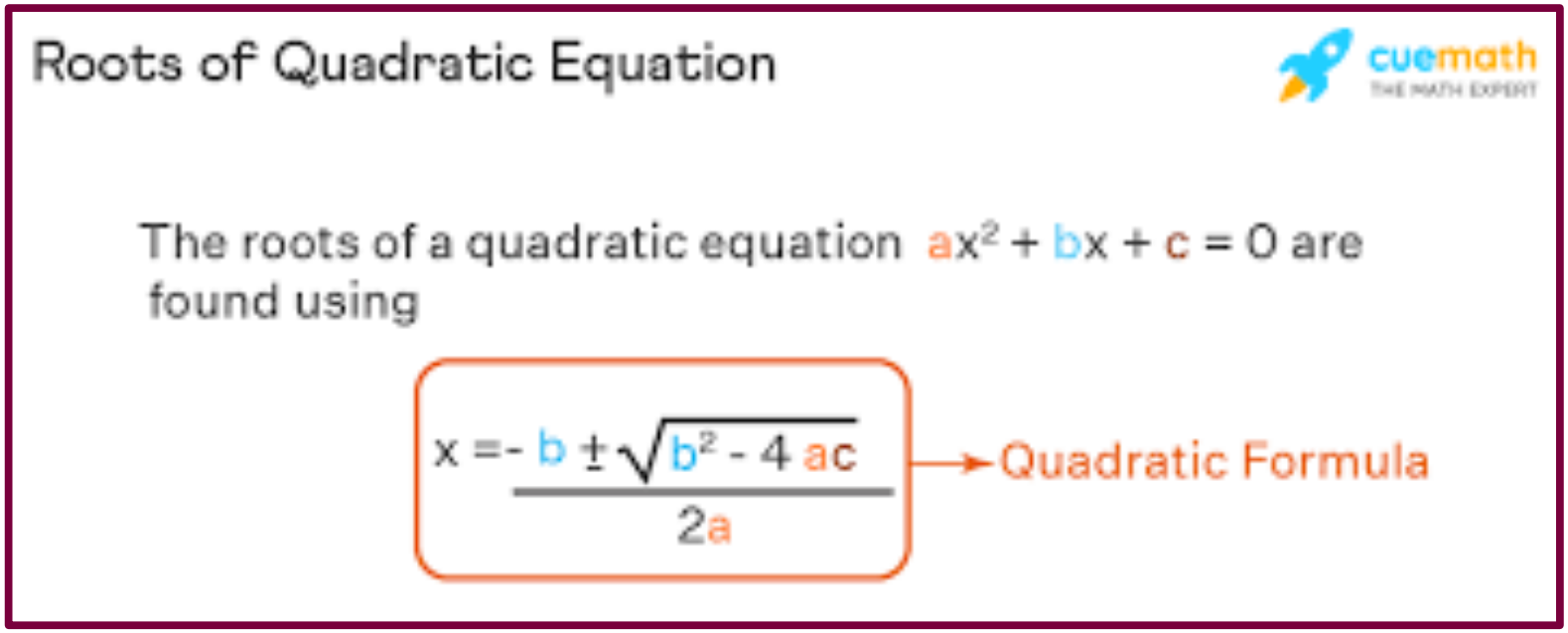


Figure 8.8

Formal Specification in the Software Process

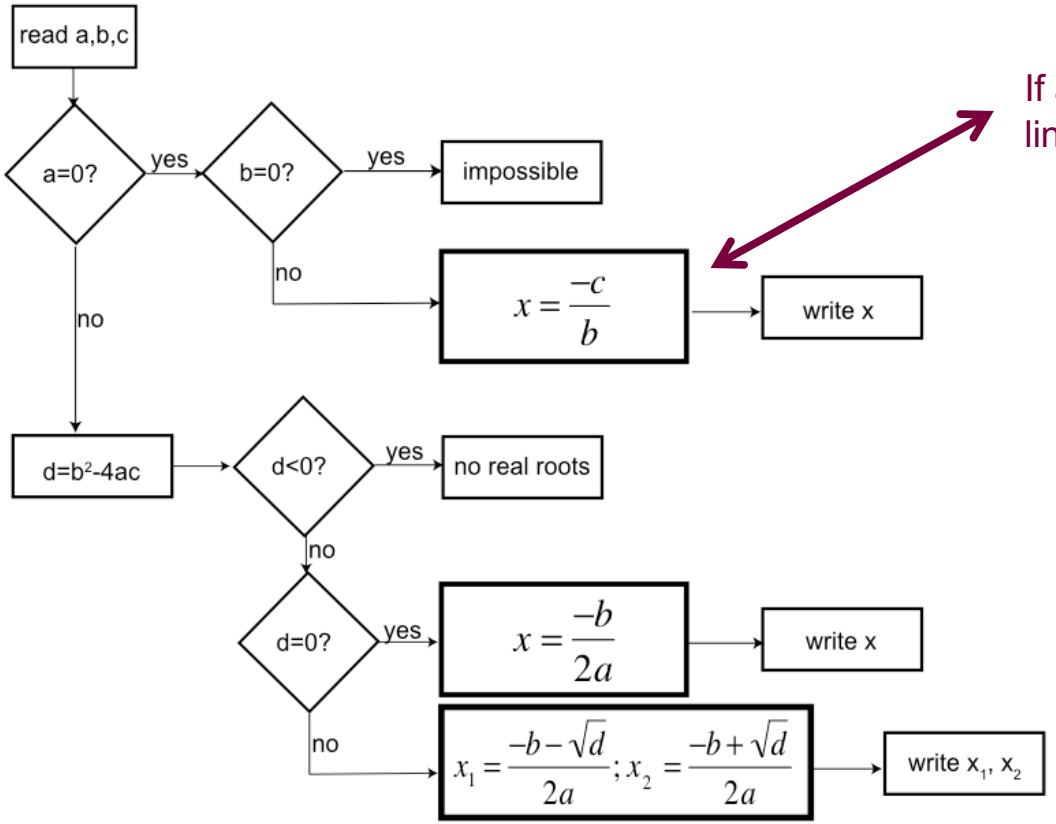
Example: Solving a Quadratic Equation



https://www.cuemath.com/algebra/roots-of-quadratic-equation/

Example: Flowchart

$ax^2+bx+c=0$



If a=0, it's a linear equation!

Floating-point problems?

Example: Pseudocode

```
if (a=0 & b=0) then solution_caption := “Ill-conditioned”
else if (b≠0) then
   solution_caption := “One root”; x1 := -c/b
else
   d := b2-4ac
   if (d<0) then solution_caption := “No real roots”
   else if (d=0) then
            solution_caption := “One real root”; x1 = -b/(2a)
   else
            solution_caption := “Two roots”
            if (b≥0) then x1 := (-b-√d)/(2a)
            else x1 := (-b+√d)/(2a)
            x2 := c/(ax1)
```

Is this a *good* specification?
Is it easy to understand?
Is this a **requirement** or is it a **design** specification?
What is the difference?

Example: Solving a Quadratic Equation

- Is this a good specification?
  - Not really. It is too detailed. It is mixing up *what* needs to be achieved with *how* it should be done.
- Is it easy to understand?
  - Yes, for engineers, but not for non-technical people
- Is this a requirement or is it a design specification?
  - Design! It explains how to compute the solution
- What is the difference?
  - *Requirements specifications* describe required behaviour in a system (WHAT)
    - *"The calculator shall determine and report the number of real solutions of a quadratic equation and provide their values if they exist."*
  - *Design specifications* describe the behaviour prescribed in a design (HOW)
    - *"If discriminant d < 0 then return 'no real roots'. Else compute roots using x = (-b ± √d) / (2a)."*

Example: Tabular Expression

			Solution caption	x_1	x_2
$a = 0$	$b = 0$		Ill defined	—	—
	$b \neq 0$		One root	$-\frac{c}{b}$	—
$a \neq 0$	$b^2 - 4ac < 0$		No real roots	—	—
	$b^2 - 4ac = 0$		One root	$-\frac{b}{2a}$	—
	$b^2 - 4ac > 0$	$b \geq 0$	Two roots	$\frac{-b-\sqrt{b^2-4ac}}{2a}$	$\frac{c}{ax_1}$
		$b < 0$	Two roots	$\frac{-b+\sqrt{b^2-4ac}}{2a}$	$\frac{c}{ax_1}$

What are the differences?

	Flowchart	Pseudocode	Tabular Expression
Differences	Shows the control flow graphically, visualize decision paths	Shows the logic in a programming-like style	Shows the conditions and outcomes clearly and compactly
Strengths	Easier for many people to follow, highlights paths and edge cases	Close to implementation, unambiguous for developers	Excellent for coverage, you need to list all possible conditions and outcomes. Compact, easy to check consistency
Weaknesses	It can get messy for complex logic	Looks like design, not requirements. Hard for non-technical stakeholders	Can grow big if many conditions

Example: Simple Control Interaction – Pseudocode

The operator in a real-time application requests or cancels a setpoint, **c_sp**, by pressing momentary pushbuttons. There are two pushbuttons, called **m_pbON** and **m_pbOFF**. They can take on the values **e_pressed** and **e_notPressed**. We specify the required behaviour as follows:

```
if (m_pbON = e_pressed ^ m_pbOFF = e_notPressed)
        c_sp = e_requested
else if (m_pbON = e_notPressed ^ m_pbOFF = e_pressed)
        c_sp = e_cancelled
```

Any problems with this?
What is both push buttons are not pressed? Or are pressed at the same time?

No specified behaviour! Then, there would not be a change in the current **c_sp** value.
It won't be continuous!

Example: Tabular Expression

Clear, unambiguous and complete!

Condition	*Result* **c_sp**
(m_pbON = e_notPressed) & (m_pbOFF = e_notPressed)	No Change
(m_pbON = e_notPressed) & (m_pbOFF = e_pressed)	e_cancelled
(m_pbON = e_pressed) & (m_pbOFF = e_notPressed)	e_requested
(m_pbON = e_pressed) & (m_pbOFF = e_pressed)	e_requested

Tabular Expressions

- A general, very simple, tabular expression:

Condition	*Result* **function name**
condition 1	result 1
condition 2	result 2
…	…
condition n	result n

- This is equivalent to:
  - if (condition 1) then function name = result 1
  - else if (condition 2) then function name = result 2
  - else if …
  - else if (condition n) then function name = result n
- Disjointness: No conditions in the table overlaps (cond i $\wedge$ cond j $\Leftrightarrow$ FALSE $\forall$i,j = 1,..,n, i $\neq$ j)
- Completeness: The conditions together cover all possible input cases (cond 1 $\vee$ cond 2 $\vee$ … $\vee$ cond n $\Leftrightarrow$ TRUE)

Function Tables

- Tabular expressions describe relations through pre and post conditions - ideal for describing behaviour without sequences of operations
- They make it easy to ensure input domain coverage
- They are easy to read and understand (you need just a little practice to write them)
- Coding from tables results in extremely well-structured code
- They facilitate identification of test cases
- Extremely good for real-time/embedded systems
- **Disadvantages**:
  - We don't know of any way of using tabular expressions to describe typical algorithms.
    - For example, how could we use a table to describe the Gauss Elimination algorithm?
  - As yet, we don't know how to use tables to document the requirements for a GUI, etc.

Side Note: Events vs. Conditions

- Events can be viewed as “pulses” in time - they do not last (retain their values) – we have referred to this as *time-discrete*



- Conditions may retain their values indefinitely – we have referred to this as *time-continuous*



Specifying Complete Programs

- A property, or requirement, for P is specified as a formula of the type

$$\{Pre\ (i_1, i_2, \ldots, i_n)\ \}$$
$$P$$
$$\{Post\ (o_1, o_2, \ldots, o_m, i_1, i_2, \ldots, i_n)\}$$

Pre: precondition, Post: postcondition

Specifying Complete Programs – Example

- Division

{exists $z(i_1 = z * i_2)$}

P

{$o_1 = i_1 / i_2$}

Specifying Complete Programs – Example

- Division – a stronger requirement

$\{i_1 > i_2 > 0\}$

P

$\{i_1 = i_2 * o_1$ **and** $o_1 \geq 0$ **and** $o_1 < i_2\}$

Specifying Classes

- When moving from individual procedures to classes, we need to ensure invariants are always respected.
  - Invariant predicates
  - Pre/Post conditions
- Example of invariant specifying an array, IMPL, used to implement an abstract data type SET
  - **for all** i, j ($1 \leq i \leq$ length **and** $1 \leq j \leq$ length **and** $i \neq j$)
    - **implies** IMPL[i]≠IMPL[j]
    - (no duplicates are stored)

Specifying Classes

In general, for every operation (op) in the class:

{Invariant and pre-conditions}
Program for op
{Invariant and post-conditions}

Summary: Conditions and Invariants

- A **pre-condition** is what must hold when an operation is called. It is the caller's obligation.
- A **post-condition** is what the operation guarantees on return, in terms of the inputs and the prior state.
- The form you saw for whole programs applies to a single operation:

```
{ Pre:  state = Programming, rate > 0, drug is in the library }
   startInfusion(drug, rate)
{ Post: rate <= hard(drug)  =>  state = Infusing and deliveredRate = rate
        rate >  hard(drug)  =>  state = Programming and verdict = rejected }
```

- A **state invariant** holds in every state, before and after every operation. It is what the module must never break.
- Pre/post conditions describe one call. A function table describes one operation across its whole input domain. An invariant describes the module as a whole.
  - D2 wants all three for the safety-critical functions.

Invariants Examples

Invariant Example	Why it matters
`state = Infusing => door closed and a valid order loaded`	The safety property: no delivery without authorization
`0 <= deliveredVolume <= orderedVolume`	Nothing over-delivers, even across pause and resume
`alarmActive => motor stopped`	Ties a controlled variable to a state
`every state change produces exactly one log entry`	Makes the NFRs checkable

Exercise: Specify one safety-critical function

1. Pick one function of your system that could harm someone if it misbehaved.

   E.g. `resumeInfusion()`
2. Write its function table: every input condition, disjoint and complete.
3. Write its pre-condition and post-condition.
4. Write one invariant your module must never break.
5. Swap with the person next to you: can they find an input case your table does not cover?

Exercise: Specify one safety-critical function

Condition	State	Motor
`state ≠ Paused`	No change	No change
`state = Paused ∧ ¬doorClosed`	Paused (verdict = rejected, DoorOpen)	stopped
`state = Paused ∧ doorClosed ∧ remaining = 0`	Complete	stopped
`state = Paused ∧ doorClosed ∧ remaining > 0 ∧ occluded`	Alarm	stopped
`state = Paused ∧ doorClosed ∧ remaining > 0 ∧ ¬occluded`	Infusing	running at orderedRate

Exercise: Specify one safety-critical function

```
{ Pre: state = Paused }
   resumeInfusion()
{ Post: doorClosed ∧ remaining > 0 ∧ ¬occluded => state = Infusing ∧ motor = running
                                                  ∧ deliveredRate = orderedRate
        doorClosed ∧ remaining = 0             => state = Complete ∧ motor = stopped
        doorClosed ∧ remaining > 0 ∧ occluded => state = Alarm    ∧ motor = stopped
        ¬doorClosed                            => state = Paused   ∧ motor = stopped
                                                  ∧ verdict = rejected(DoorOpen)}
```

- Invariant: `state = Infusing ⟹ doorClosed ∧ ¬occluded ∧ validOrderLoaded`

Deliverable 2 Map

D2 Document Section	W5 L1	W5 L2	W5 L3	W6
1. Introduction and context	-	-	-	-
2. Precise specification	✓	X		
3. Modular design and interfaces		X	X	
3.1. External ADC <-> Monitoring interface				X
4. Implementation notes				X
5. Traceability table				X
6. Assumptions and decisions			X	X

SFWRENG / MECHTRON 3K04 Software Development

Today: The 4-variable model

McMaster University
Engineering
Computing & Software

October 8th, 2026

Dr. Angela Zavaleta Bernuy

Plan Review: Requirements & Validation

- If we don't get the requirements right, it does not matter how good we build our system.
- The world of *requirements*, can be divided into a few parts:
  - **Eliciting** requirements: how do we **capture** the requirements
  - **Documenting** requirements: how do we **keep track** of the requirements
    - **Formal specifications**: well-defined syntax and semantics (usually mathematical)
    - **The 4 variable model**: relationship between requirements and design ← *** WE ARE HERE ***
  - How do we know if we elicit and document requirements **correctly**?
- *Validation*: determining whether or not we got the *right* requirements
  - Not to be confused with *verification*, which is determining whether or not we built the system *right* (i.e., if it complies with the requirements)
  - We often refer to Validation and Verification as V & V.

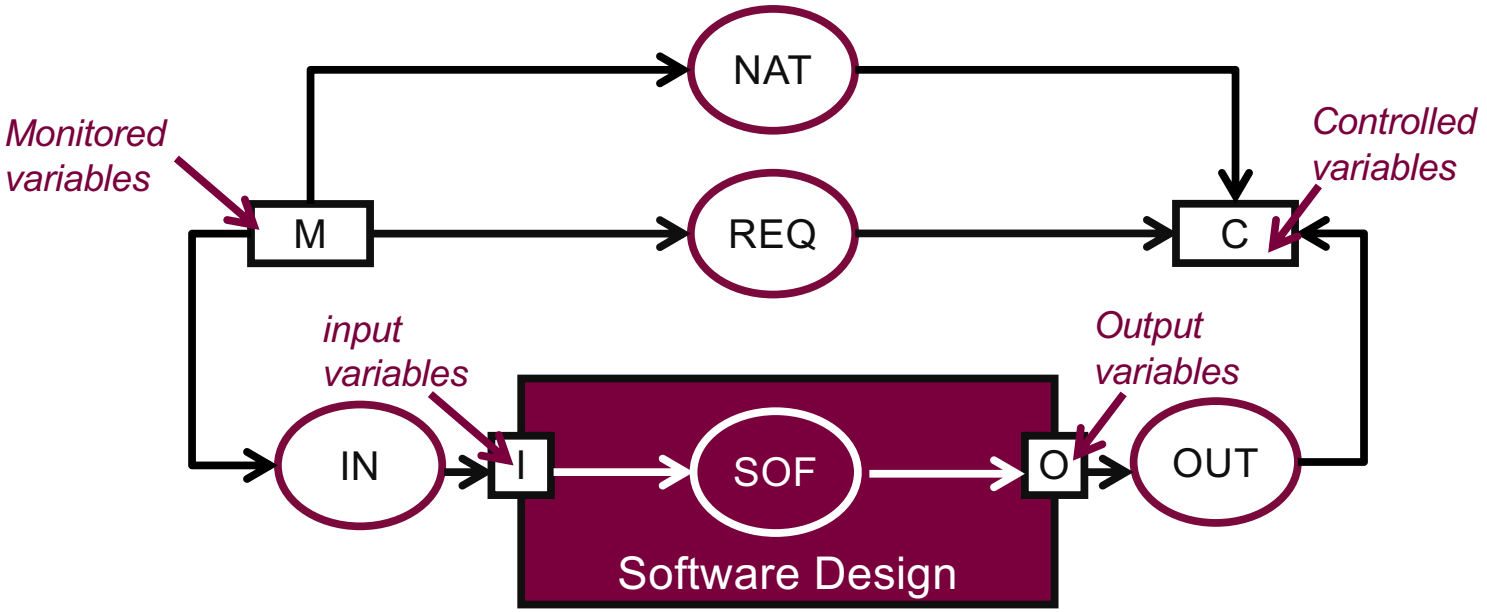
The 4 Variable Model

- Describes the essential relationship between requirements and design of embedded and safety-critical systems
- Illustrates why the software design has different input and outputs from the requirements
    - I.e. Way to keep requirements (what the system must achieve in the environment) separate from implementation details (how we make that possible)
- It is often used to show the relationship between a system and the real world.
- Parnas, D.L., Madey, J.: Functional documents for computer systems. Science of Computer Programming 25 (1995) 41-61

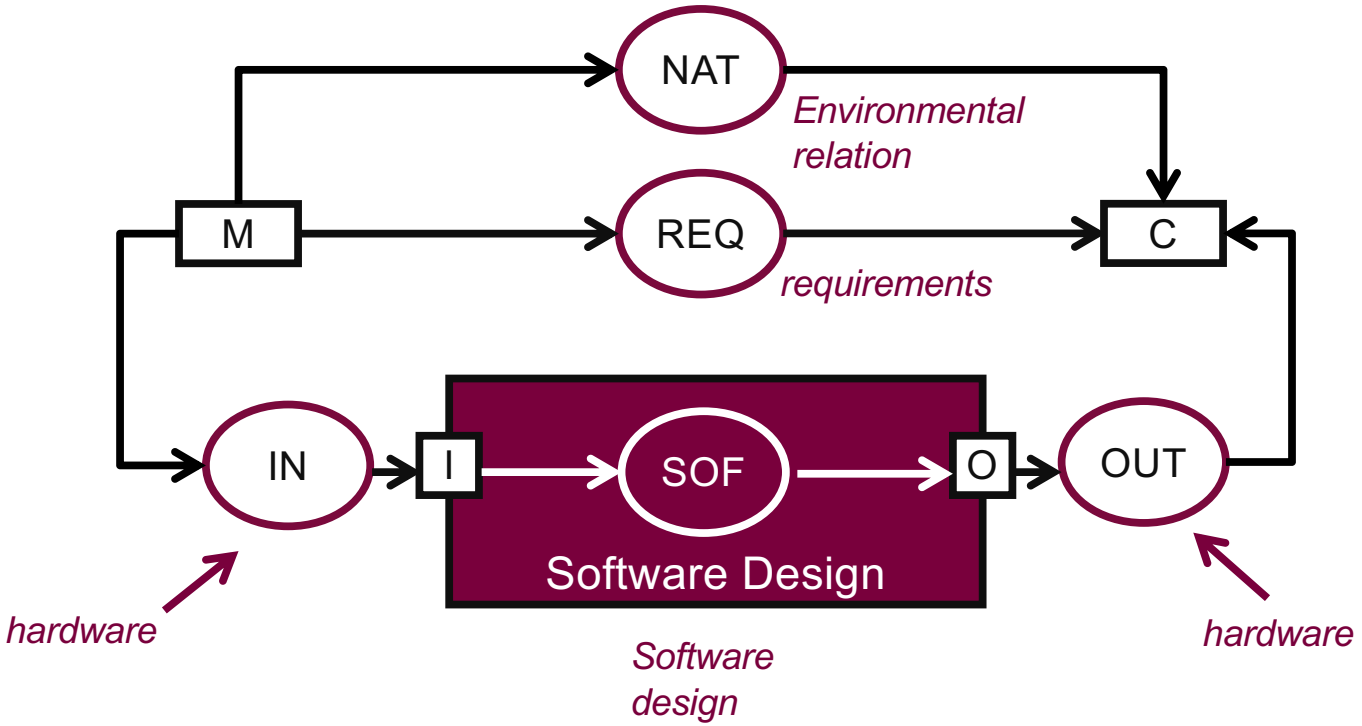
The 4 Variable Model

- Monitored variables (M): Properties in the environment that the software needs to watch
  - E.g., button pressed
- Controlled variables (C): Properties in the environment that the software is supposed to control
  - E.g., setpoint requested
- Input variables (I): What the software *receives* from sensors
  - Representations of M
- Output variables (O): What the software *sends* to actuators
  - Representations of C

The 4 Variable Model: Variables

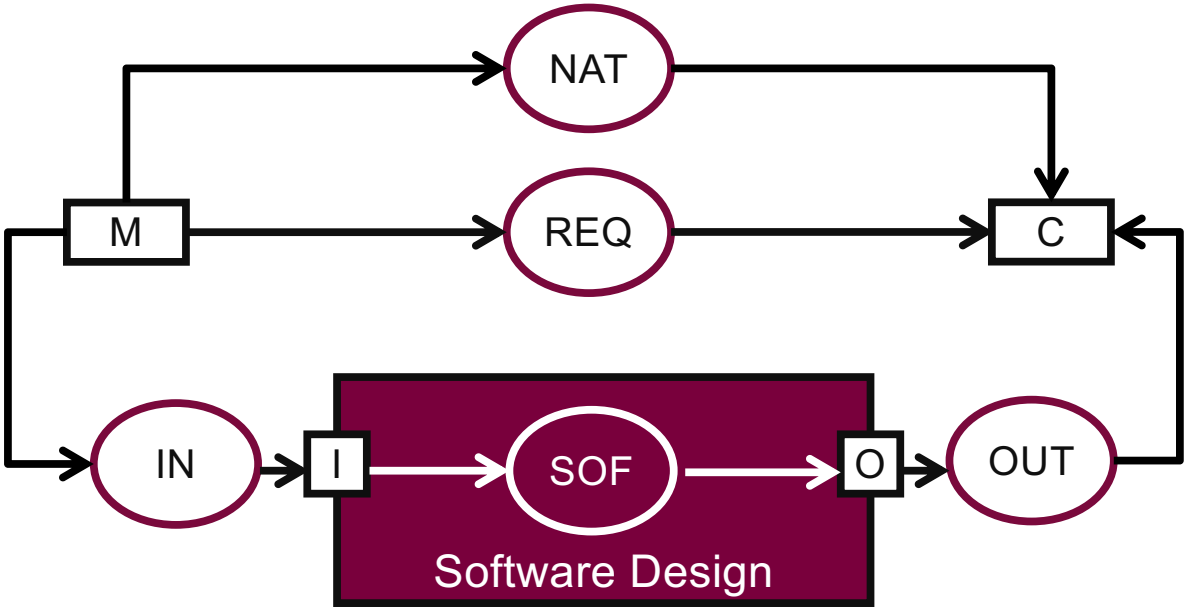


The 4 Variable Model: Relations/Functions



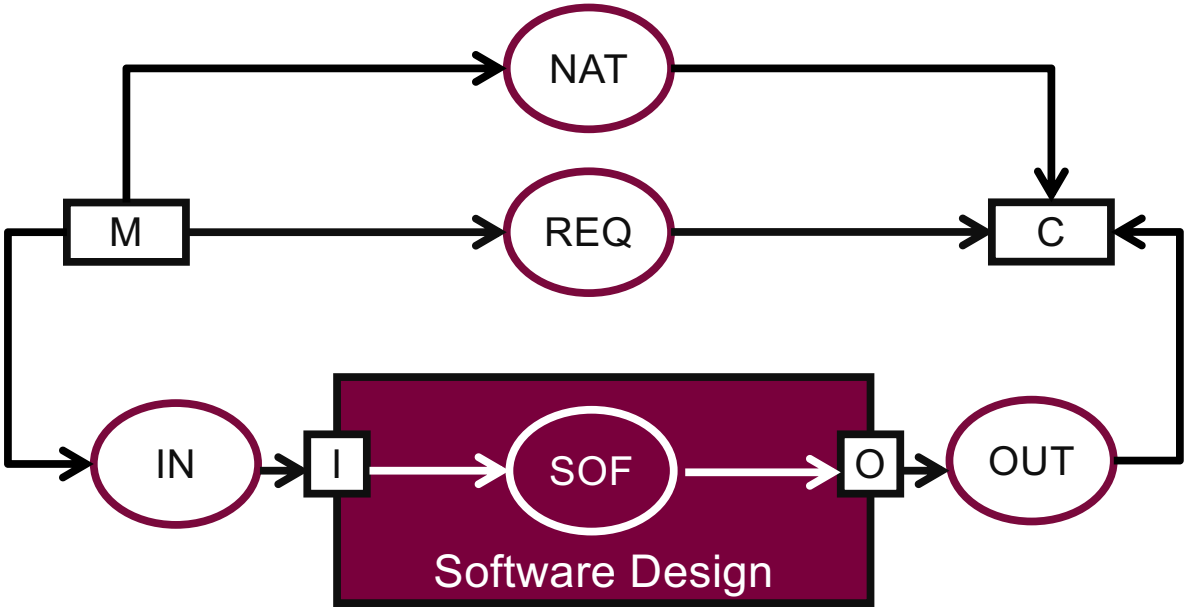
The 4 Variable Model: Formal Relationships

- IN (sensor model)
  - How real-world monitored values are turned into software inputs
- OUT (actuator model)
  - How software outputs affect the environment
- SOF (software design)
- REQ (requirement)
  - What we *want* the system to achieve
- NAT (natural laws)
  - How the environment *behaves*



The 4 Variable Model: Formal Relationships

- C = REQ(M)
  - The requirements describes the intended relationship between the environment and variables
- I = IN(M)
  - Input devices transform monitored variables into input signals
- C = OUT(O)
  - Output devices transform software outputs into real-world controlled variables



SOF: I -> O such that OUT(SOF(IN(M))) = REQ(M)

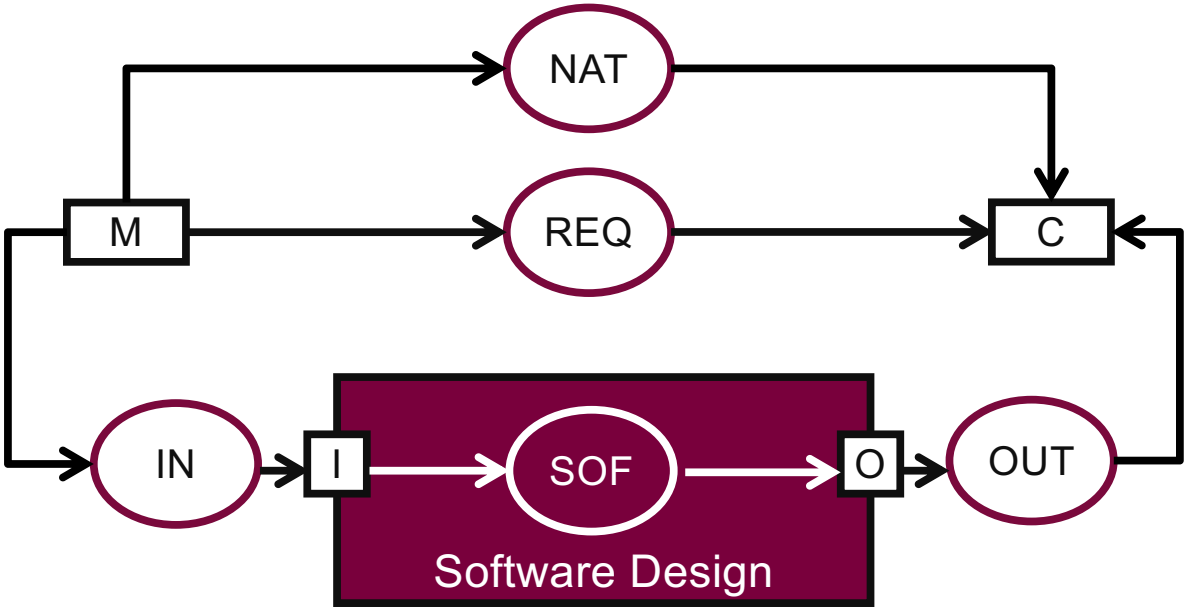
The 4 Variable Model: Example

Assume we have a sensor that converts physical temperature in Celsius to an unsigned 12-bit integer.

We can also determine that *i* = round(*m**4/5), where *m* is the monitored variable (temperature) and *i* is the 12-bit integer that represents m in the software.

Example: if the room is 25 C, the sensor IN will convert that into *i* = 50.

The software will never see “25 C”, only the encoded integer.

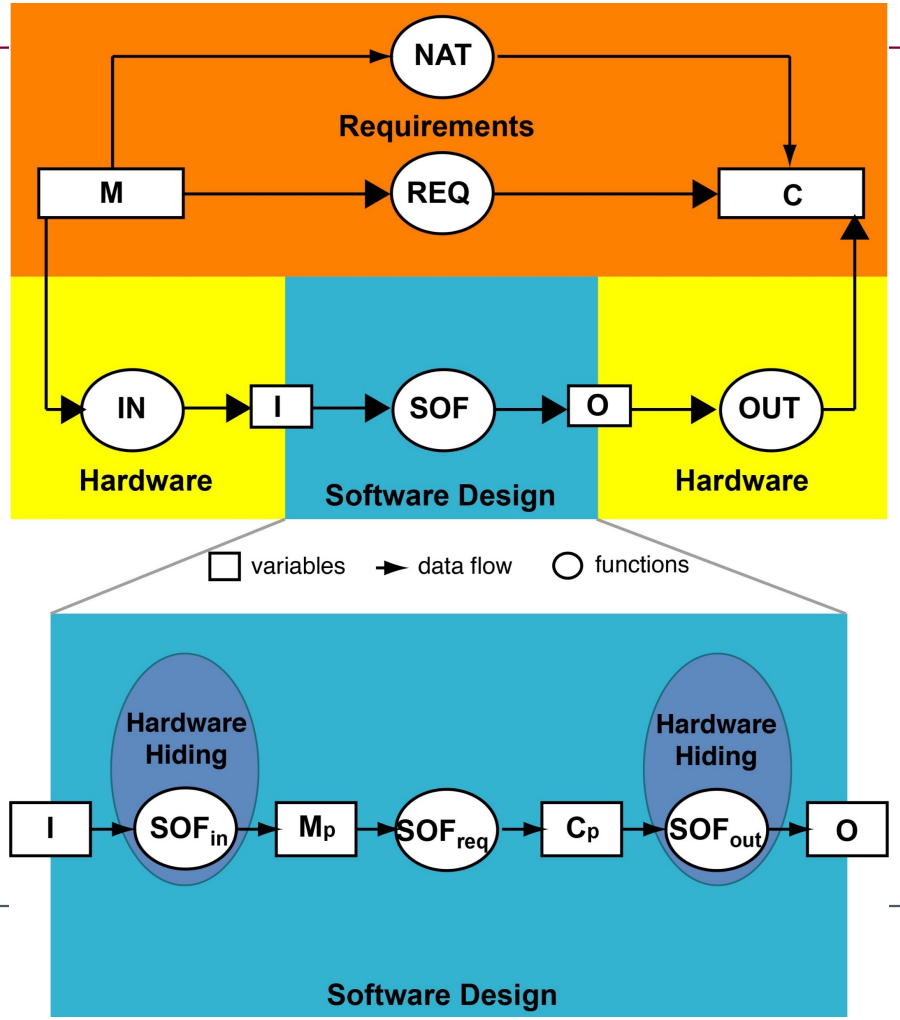


The 4 Variable Model in Safety-Critical Systems

- Regulators care about real-world requirements (M and C)
  - REQ is stated in terms of M and C
- Software engineers must translate these into implementations using I and O, while documenting assumptions about IN and OUT mappings
  - SOF works with I and O
- If the mapping is ignored, you risk dangerous mismatches
- TLDR:
  - Write requirements in terms of monitored (M) and controlled (C) variables
  - Write design in terms of input (I) and output (O) variables
  - Explicitly document the IN and OUT mappings (assumptions about sensors and actuators)
- We have to remember that in software, we use i (in I) instead of m (in M).

The Modified 4 Variable Model

- M_p and C_p are called pseudo-M and pseudo-C: software-usable versions of those variables
- SOF_{in}: Converts raw input variables I into a software-usable representation M_p
- SOF_{req}: Works on M_p and decides what the controlled properties C_p should be
- SOF_{out}: Converts the desired controlled property C_p into the actual actuator O
- Hardware hiding: Encapsulates the details of I and O, isolating the rest of the software from hardware-specific details.
  - This makes the design portable and testable (you can simulate M_p and C_p without real hardware)



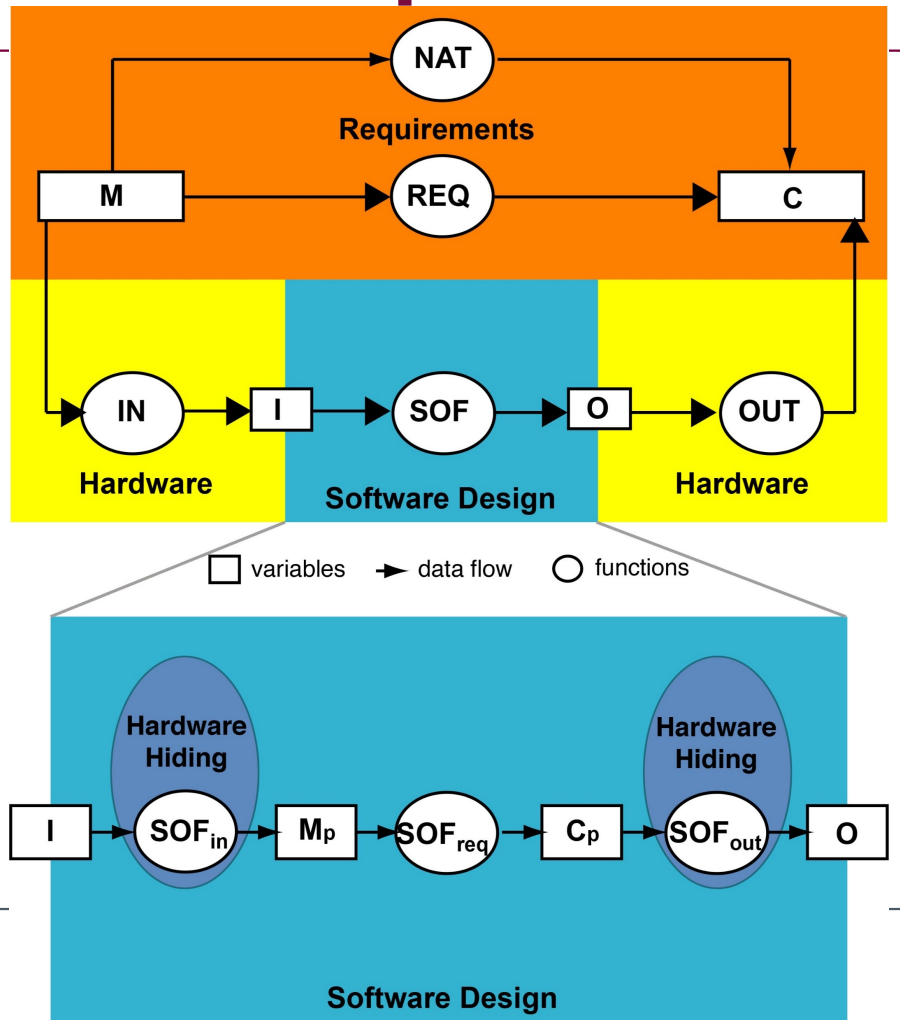
The Modified 4 Variable Model: Example

Assume we have a sensor that converts physical temperature in Celsius to an unsigned 12-bit integer

We can also determine that i = round(m*4/5), where m is the monitored variable (temperature) and i is the 12-bit integer that represents m in the software.

If we want to produce m_p so that it is close to the original m, we can multiple i by 5/4, so we get: $m_p = i*5/4$

$| m - m_p |$ < error bound



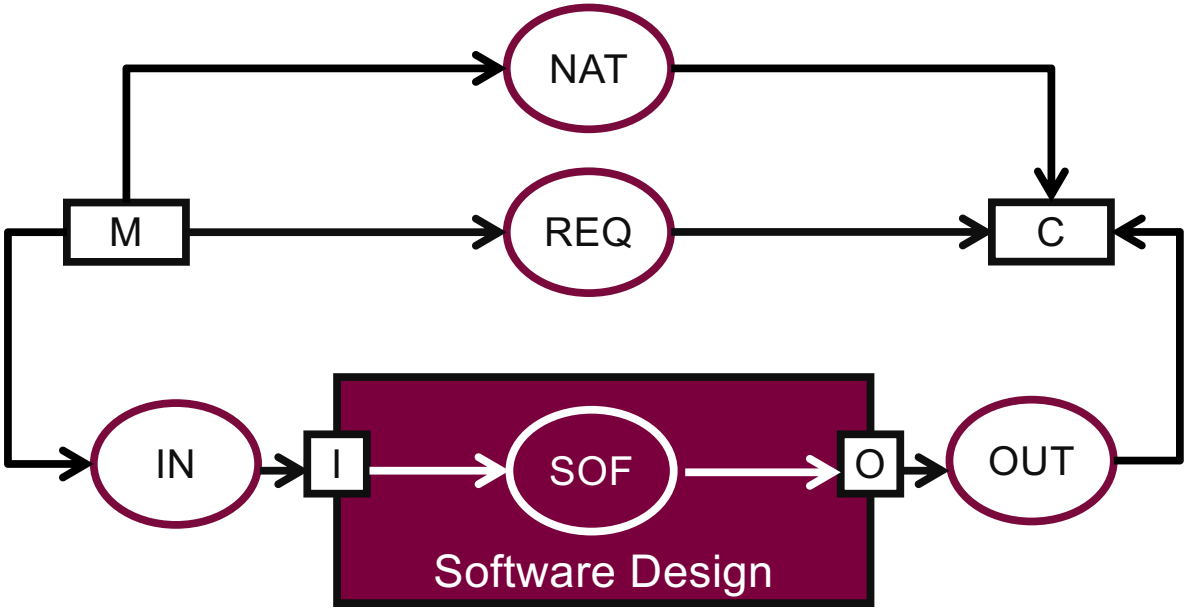
Time for a small example

Pickering Nuclear Generating Station



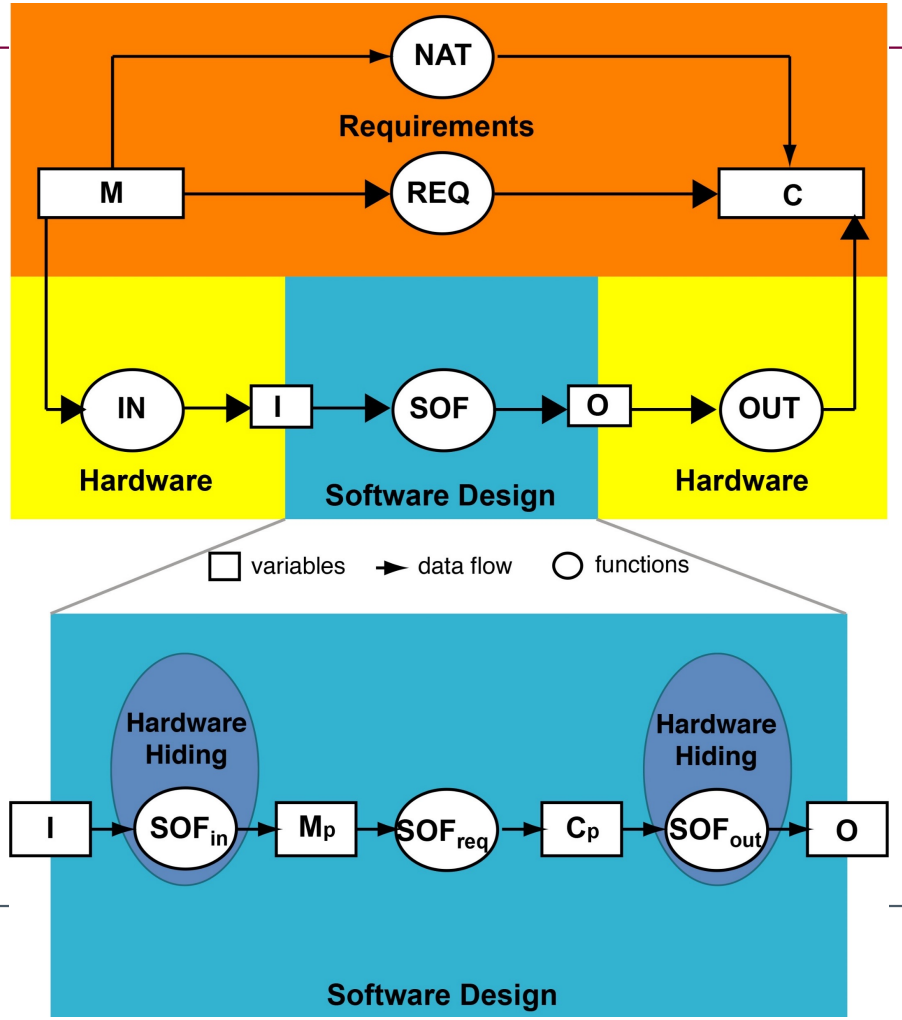
PNGS: The 4 Variable Model

- REQ: If the coolant temperature is above 300 C, the rod position must be inserted.
- M: actual temperature of the coolant
- C: either withdrawn or inserted
- I: ?
  - IN has to convert the temperature into a digital signal
- O: 0/1 ?
  - OUT will convert the value into either withdrawn or inserted

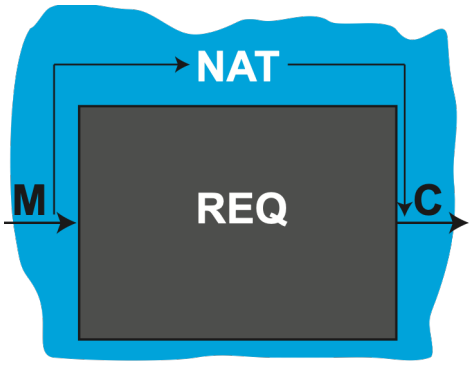


PNGS: The Modified 4 Variable Model

- SOF_in: Will translate the raw input to a meaningful physical quantity M_p
- SOF_req: Implements the requirements using M_p
  - C_p will get the value of inserted if M_p is greater than 300, and withdrawn otherwise
- SOF_out: Will receive the value of 0/1 based on C_p's value
- Why?!
  - The software can't see M directly
  - Sensors are imperfect
  - Separation helps with V&V



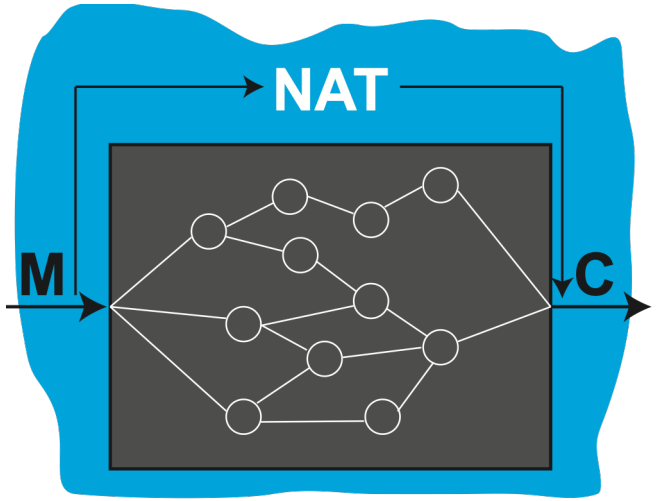
System Requirements: Review



We need to specify REQ in terms of M and C without any intermediate functions.
This would give us a black-box description of the required behaviour.

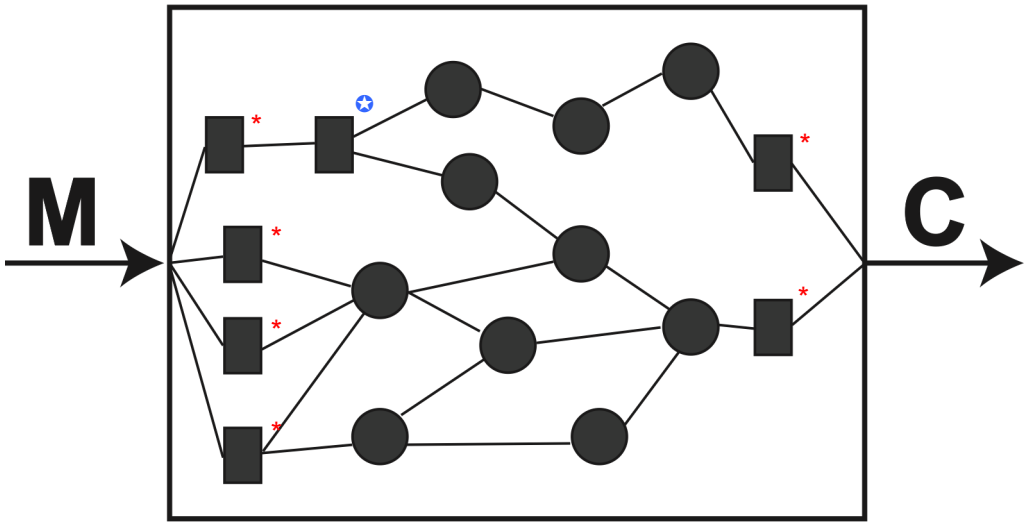
$$C \in REQ(M)$$

This is not always possible.
The behaviour is too complex to describe it in a single function/relation.
We use intermediate functions to describe behaviours that still make sense to the domain expert.
The overall behaviour is given by the composition of the functions.



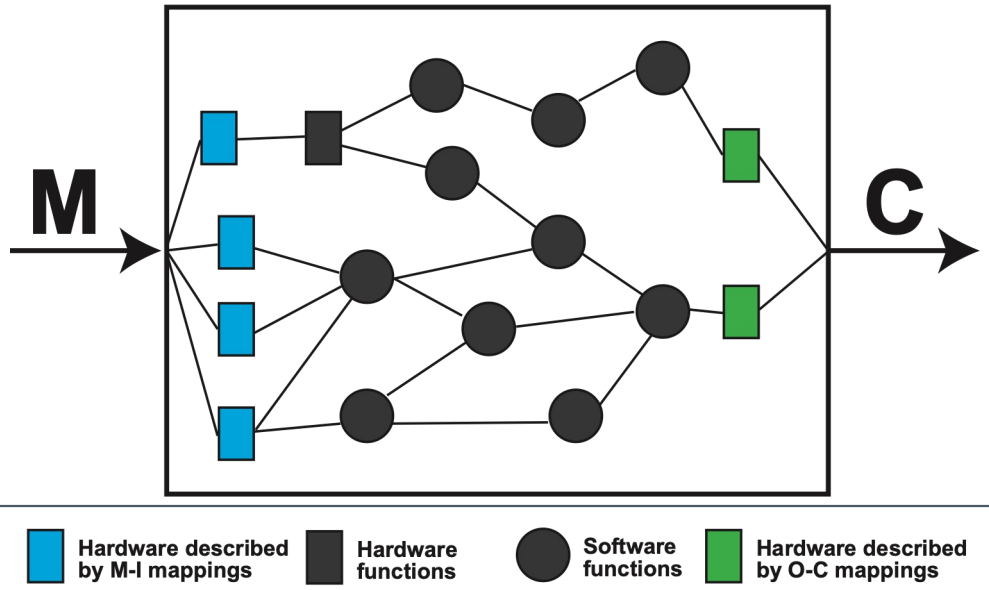
System Design: Overview

- Once we have the behaviour specified in the system requirements, we can partition the behaviour into hardware and software functions.
- In most cases, the hardware components will represent sensors and actuators and associated hardware represented by IN and OUT in the 4 variable-model*.
- In a few cases, a hardware component may perform actual functional behaviour ✪.
- The software functions in the system design can be used to specify the software requirements, if they are described adequately in the system design.



System Design: Overview

- Once we have the behaviour specified in the system requirements, we can partition the behaviour into hardware and software functions.
- In most cases, the hardware components will represent sensors and actuators and associated hardware represented by IN and OUT in the 4 variable-model.
- The hardware functions that represent IN and OUT must be described by M-I and O-C mappings.



Examples: Monitored -> Input (IN)

Monitored	Quantity, range	Input device	Software sees
`m_doorPosition`	{open, closed}	door microswitch	`i_doorSwitch: bool`
`m_linePressure`	0–800 mmHg	transducer + external ADC	`i_pressureCount: 0..4095`
`m_keyON`, `m_keyOFF`	{pressed, notPressed}	keypad	`i_keyON`, `i_keyOFF: bool`
`m_time`	seconds	timer	`i_tick: ms`

Examples: Output -> Controlled (OUT)

Software writes	Output device	Controlled	Quantity
`o_motorStepRate`	stepper driver	`c_drugFlowRate`	0.1–999 mL/h into the patient
`o_buzzer: bool`	piezo	`c_audibleAlarm`	≥ 65 dB at 1 m
`o_displayBuffer`	LCD	`c_displayedStatus`	what the clinician reads
`o_logWrite`	flash	`c_auditTrail`	one record per state change

Deliverable 2 Map

D2 Document Section	W5 L1	W5 L2	W5 L3	W6
1. Introduction and context	-	-	-	-
2. Precise specification	✓	✓		
3. Modular design and interfaces		✓	X	
3.1. External ADC <-> Monitoring interface				X
4. Implementation notes				X
5. Traceability table				X
6. Assumptions and decisions			X	X

SFWRENG / MECHTRON 3K04 Software Development

Today: Module design and Information hiding

McMaster University | Engineering
Computing & Software

October 9th, 2026

Dr. Angela Zavaleta Bernuy

What do we know so far?

- Requirements tell us what the system must do.
- Great!
- But, **how do we structure the system so it is understandable, testable, and maintainable?**
- Recall our Pickering Nuclear Plant example
  - From the requirements, we learnt that if the coolant is over 300 C, the shutdown rods will be dropped.
  - That is a very clear requirement!
  - We don’t know: how to organize the modules for sensors, actuators, safety checks, etc.

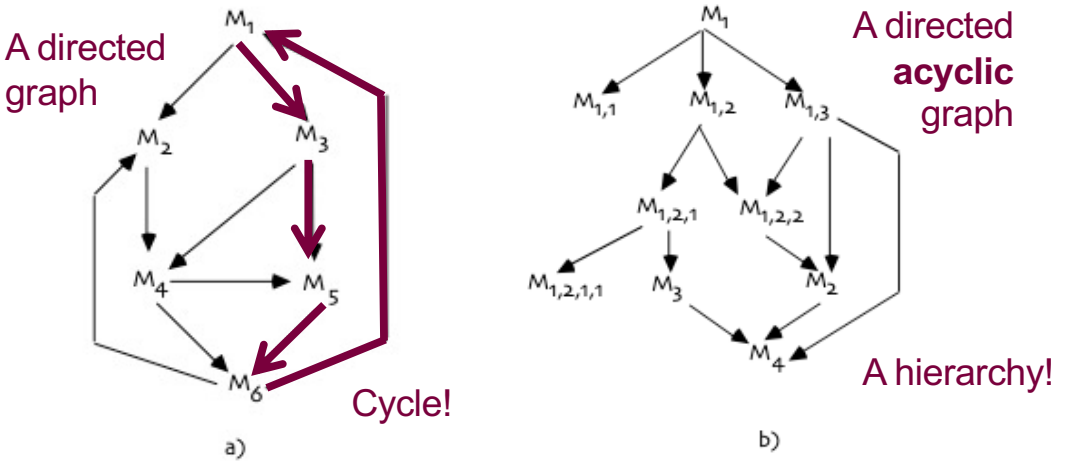
High-Level Software Engineering Concepts

- How to structure the software so the requirements can be implemented safely, clearly, and in a way regulators will accept.
- We are going to cover four main topics:

1. Relational Hierarchies
2. Cohesion and Coupling
3. Module Design
4. Module Documentation

1. Relational Hierarchies

- A way of organizing modules and components into layered or *hierarchical* structures that show how they depend on and relate to each other.
- Let's visualize relationships between modules using something we know:
  - Graphs!
  - Let M_# be a module



1. Relational Hierarchies: The USES Relation

- The fundamental dependency that organizes software into hierarchy
- Let's say, A is the client of B and B is a server.
- A **uses** B
  - A requires the correct operation/behaviour of B
  - A can access the services exported by B through its interface
  - A depends on B to provide services promised by A
- This is a desirable property!

1. Relational Hierarchies: The USES Relation

- USES should be a hierarchy
  - Clear direction of dependence
- Hierarchies make software easier to understand
  - We can proceed from leaf nodes (who do not use others) upwards
- They make software easier to build
  - You can implement the leaf modules first, since nothing depends on them
- They make software easier to test
  - Testing can also start bottom-up
- Not the only form of hierarchy in software…

1. Relational Hierarchies: Other Hierarchies

- Composition hierarchy
  - A hierarchy based on containment: one module contains or is composed of other modules
  - Allows for *encapsulation*: other modules don't need to know the internal details of others
  - Enables easier *information hiding*: details are hidden inside the components
- Inheritance hierarchy
  - A hierarchy based on class inheritance, one class is a subclass of another and inherits its properties and methods
  - Overuse can lead to bloat: subclasses filled with unnecessary attributes and methods
  - Powerful for reuse, but in critical systems, it's risky if not carefully controlled
    - For instance, standards like MISRA (in automotive software) often restrict inheritance for safety reasons

1. Relational Hierarchies: Summary

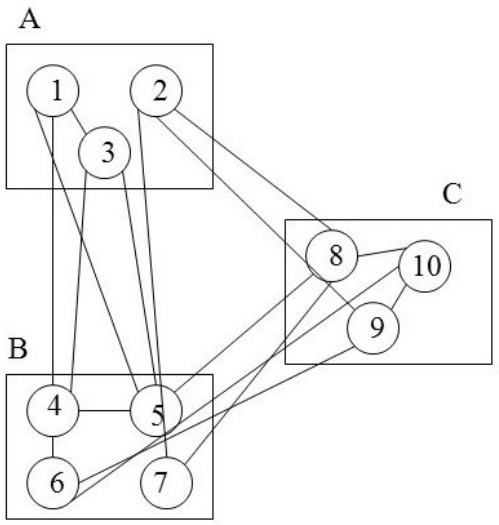
- Uses hierarchy
  - About *dependencies*, who relies on whom
- Composition hierarchy
  - About *containment*, modules made up of other modules
- Inheritance hierarchy
  - About *types* and *reuse*, modules inheriting properties from others

2. Cohesion and Coupling

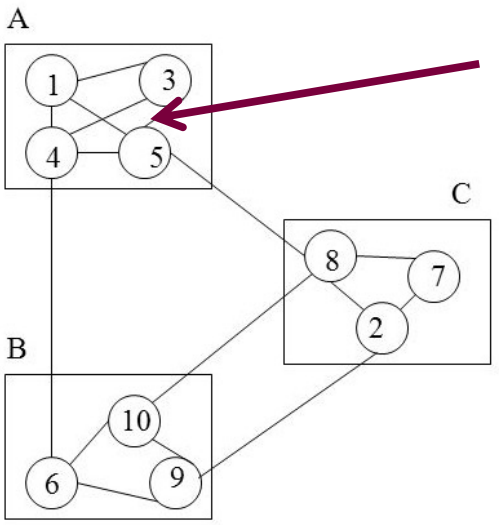
- We have talked about this before ☺
- Each module should be *highly cohesive*
  - Modules are understandable as a meaningful unit
  - Components of a module are closely related to one another
- Modules should *exhibit low coupling*
  - Modules have low interactions with others
  - Modules are understandable separately

2. Cohesion and Coupling

We can evaluate ***coupling*** from an external view ("uses"), but we need to see internal details to judge ***cohesion***.



Bad modularization:
low cohesion, high coupling



Good modularization:
high cohesion, low coupling

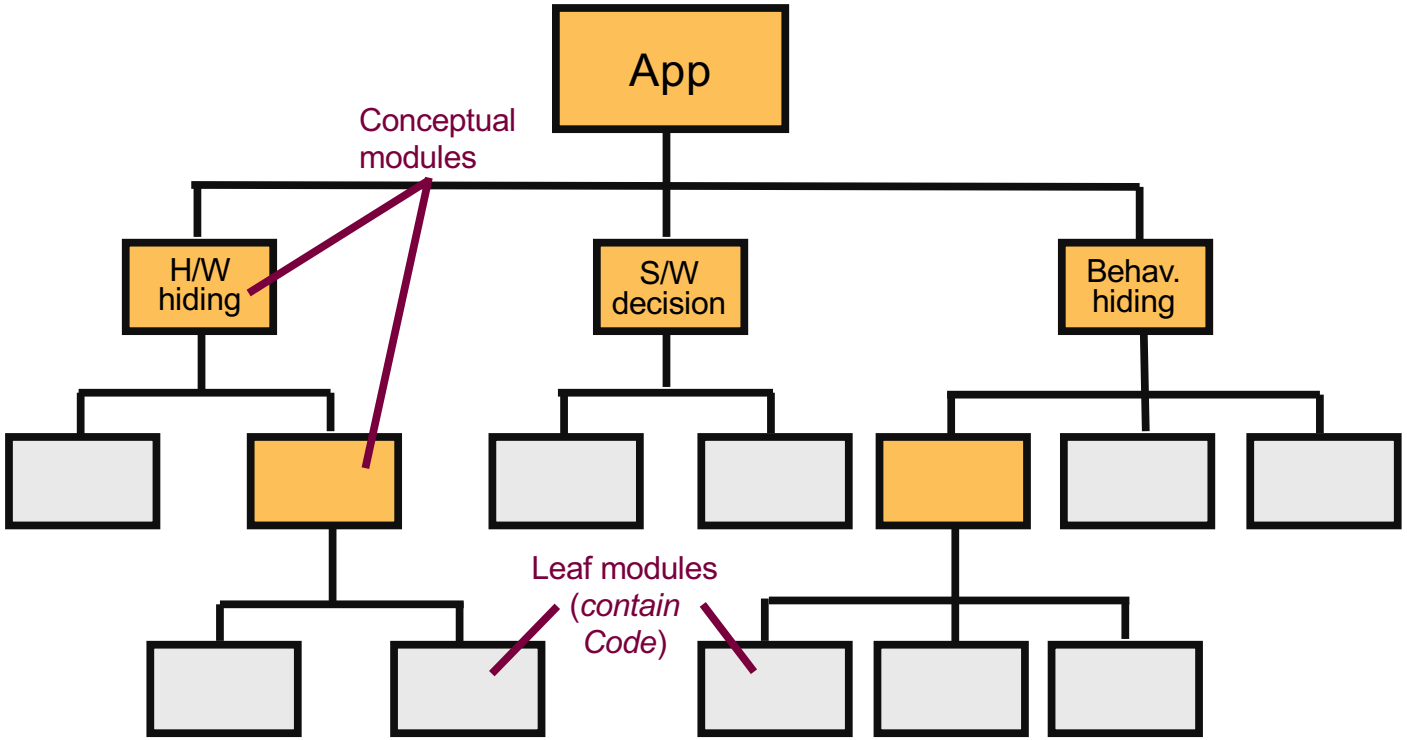
Usually an indication of reasonable cohesion

https://devopedia.org/cohesion-vs-coupling

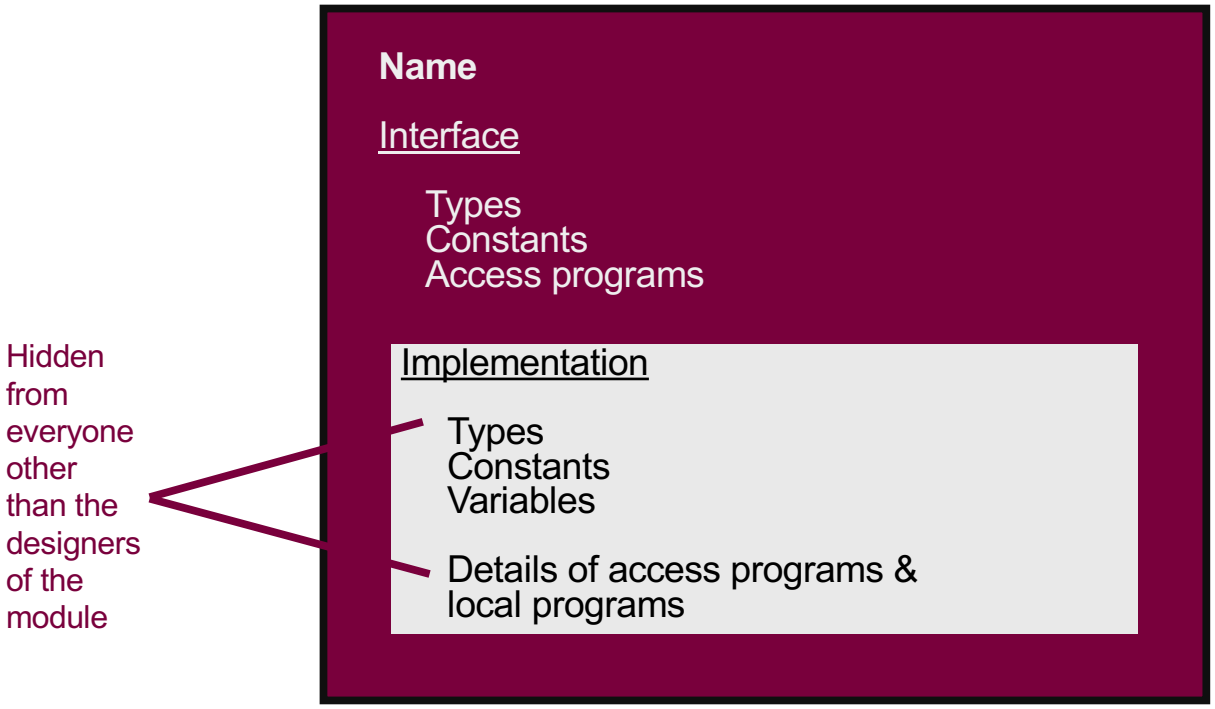
3. Module Design

- The process of defining clear boundaries, responsibilities, and interfaces for software modules
  - **Goal**: Make the system easier to develop, test, maintain, and to be reviewed by a regulator!
- Why does it matter?
  - Makes the code easier to change without breaking the whole system
  - Critical in safety-critical systems, where regulators require clear design documentation
- Key principles:
  - Information hiding: modules expose only what others need (interface) and keep details private (implementation)
  - Separation of concerns: each module is responsible for one well-defined function
  - Independence: modules should minimize unnecessary interdependencies
- How?

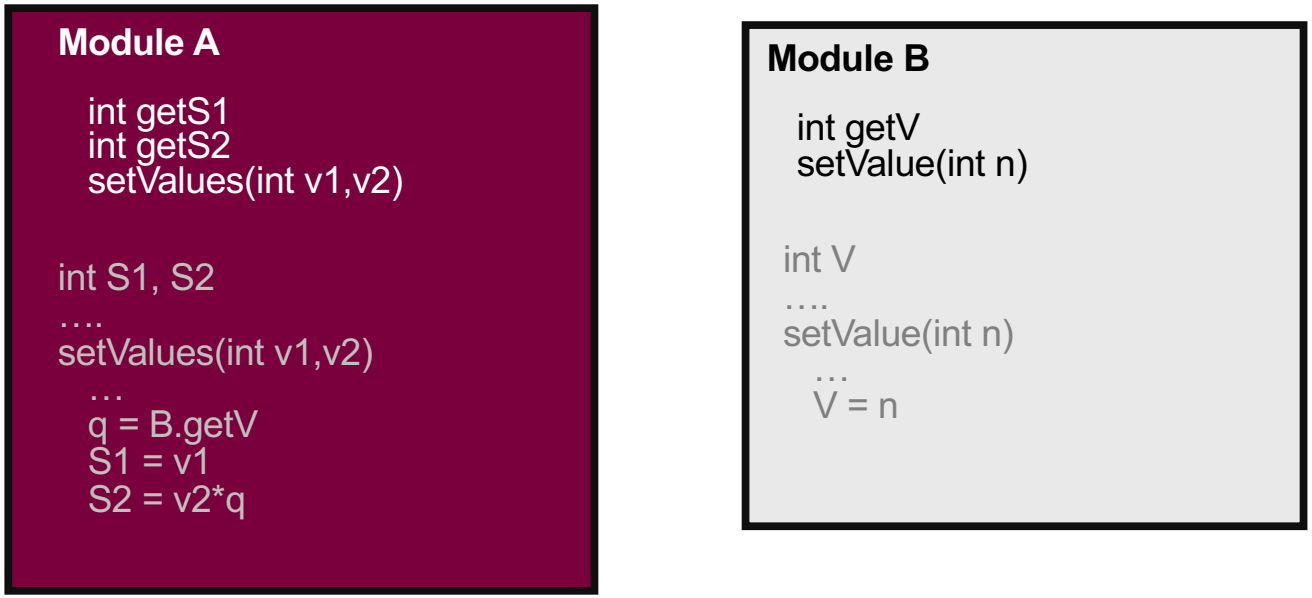
3. Module Design: Module Decomposition (Parnas)



3. Module Design: Module Fundamentals



3. Module Design: Module Fundamentals



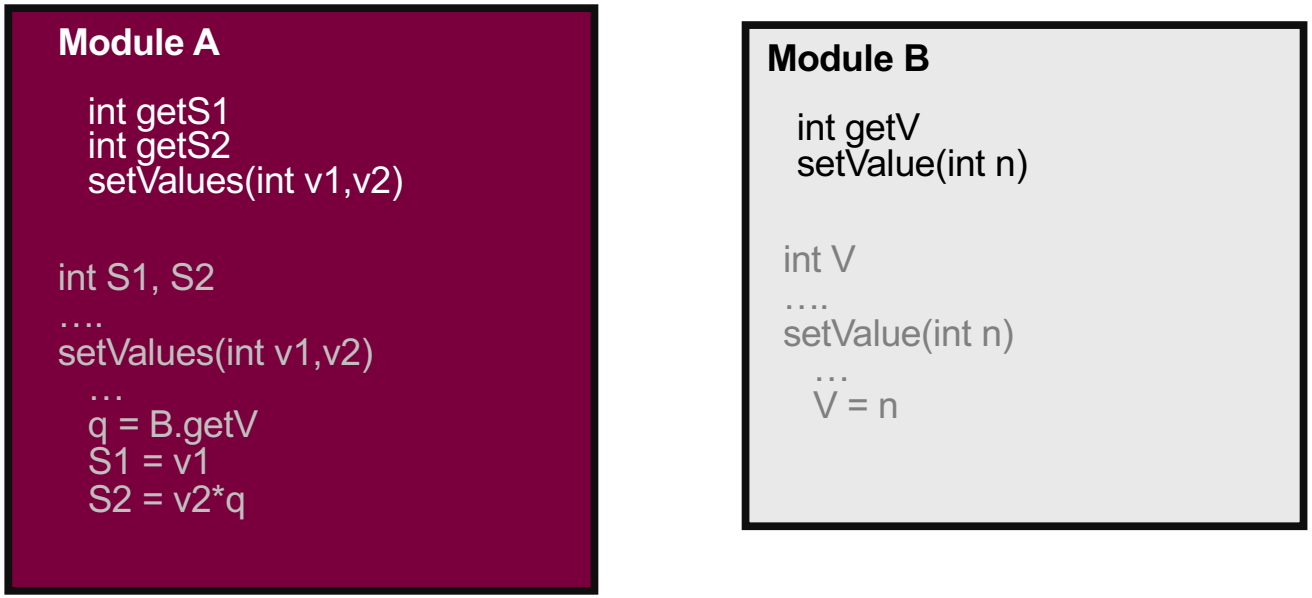
Module A uses Module B via de USES relation

A depends only on B's interface, not its implementation.

3. Module Design: Interface vs. Implementation

- To understand the nature of the USES relationship, we need to know what a used module *exports* through its *interface*
- The client *imports* the resources that are exported by its servers
- Modules *implement* the exported resources
- Implementation is *hidden* to clients
- Key design principle: clear distinction between interface and implementation:
  - Supports separation of concerns:
    - Clients care about resources exported from servers and servers care about implementation
  - The interface acts as a contract between a module and its clients

3. Module Design: Interface vs. Implementation



Module A uses Module B via de USES relation

A depends only on B's interface, not its implementation.

3. Module Design: Module Fundamentals

The **interface** is like the tip of an iceberg



The **implementation** is like the bulk of an iceberg

Need to **document the *effects* of interface functions**, not just their inputs/outputs, but also what they change

3. Module Design: Interface vs. Implementation

- To describe the interface, we need to describe the relationship between:
  - outputs (responses) and
  - inputs (stimuli)
- There are many different ways of doing this, ranging from natural language to formal notation
  - We will discuss this later!
  - Some of which we have discussed already…
  - Tables, flowcharts, Gherkin's, etc.

3. Module Design: Information Hiding

- Core design principle that is the basis for back-end design (i.e. module decomposition)
- *Encapsulate changeable (requirements) and design decisions as implementation secrets within module implementations*
- That is:
  - Implementation details are hidden to clients
  - These details can be changed freely if the change does not affect the interface (or even the entire system)
    - Hardware: Sensors, input/output devices, actuators, etc.
    - Behaviour: Number of items, timing, etc.
    - Design decisions: Data structures, algorithm, platforms, etc.
- Changes may occur in the requirements or/and in the design decisions stage

3. Module Design: Information Hiding

- How can we apply the information hiding principle?
  - Make a list of anticipated changes while preparing requirements
  - Make a list of changes that are not likely while preparing requirements
  - For each likely change, decide on the “secret” you want to hide and document it
  - As you finalize requirements and start the design, update the above lists
  - *Apply information hiding principle in developing the design*
  - At the end of design, make sure list is accurate
  - At the end of the design, “test” how well you have followed the information hiding
    - Design review test!

3. Module Design: Information Hiding

- Design review test:
  - Did you successfully **encapsulate** the parts of the system that are likely to change (e.g., hardware details, algorithms, data formats)?
  - If one of those changes happened tomorrow, would it **only require changes inside one module**, without ripple effects across the system?
  - Did you keep the interfaces clean, exposing **only what clients actually need**, and **hiding implementation** details?
  - Are the secrets you chose (the design decisions likely to change) **really hidden**, or did some of them leak out into other parts of system?

3. Module Design: Information Hiding

- Design review test:
  - Did you successfully **encapsulate** the parts of the system that are likely to change (e.g., hardware details, algorithms, data formats)?
  - If one of those changes happened tomorrow, would it **only require changes inside one module**, without ripple effects across the system?
  - Did you keep the interfaces clean, exposing **only what clients actually need**, and **hiding implementation** details?
  - Are the secrets you chose (the design decisions likely to change) **really hidden**, or did some of them leak out into other parts of system?
- To make this work, we need a clear and careful interface.

3. Module boundaries around likely changes

The single most common mistake is decomposing by process step (LoginModule, DispenseModule, LogModule) instead of by *secret*. Process steps are not what changes.

Anticipated change	Secret to hide	Module
A different occlusion sensor is fitted	How sensor readings are converted to a monitored value	Sensor hiding
The hospital switches from Wi-Fi to a wired protocol	Message format and transport	Comms hiding
The screen is replaced, or the UI is translated	Layout and wording	UI hiding
Logs move from a local file to a database	Storage format and location	Log storage
Pharmacy changes the dose-limit rules	How limits are represented and looked up	Dose policy
The regulator demands a longer alarm escalation time	The timing constants	Behaviour hiding

3. Module Design: Interface Design

- The interface should not reveal what we expect may change later
- It should act as a firewall preventing access to hidden parts
- The interface should not expose internal data structures
- The interface is what connects modules together
  - And it is where mistakes in design show up first!

3. Module Design: Prototyping

- Once an interface is defined, implementation can start
  - The first version will be done quickly, and inefficiently
  - Then, progressively turned into the final version
- The initial version acts as a prototype that evolves into the final product
- To make sure our designs are precise and understandable by others, we rely on design frameworks and notations!

A Module is not a Class (but it becomes one)

In the design (Module Guide)	In the code
Module	one class, or a small namespace of classes
Secret	`private` fields and constants, absent from the public signature
Interface / exported operations	the `public` methods, with pre/post in the doc comment
State invariant	established by the constructor, preserved by every public method
USES	the references a class holds, injected, and still acyclic

Deliverable 2 Map

D2 Document Section	W5 L1	W5 L2	W5 L3	W6
1. Introduction and context	-	-	-	-
2. Precise specification	✓	✓		
3. Modular design and interfaces		✓	✓	
3.1. External ADC <-> Monitoring interface				X
4. Implementation notes				X
5. Traceability table				X
6. Assumptions and decisions			✓	X

Everything up to this point, it is fair game for the Midterm ☺

Midterm: Friday, October 23rd 11:30 AM – 12:20 PM

- **BE ON TIME! (Arrive early)**
- Two rooms – Check Avenue announcement for where to go.
  - **T13 123**
  - **T13 127**
- 45 minutes
- One cheat sheet is allowed. Letter sized. Double-sided.
- Types of questions:
  - Multiple-choice for *theory*
  - Short answers for *practice*
    - *You practice by working on your project!*
- Tuesday, October 20th during lecture: midterm review session